

LESSON 4: BEHIND THE INTERFACE

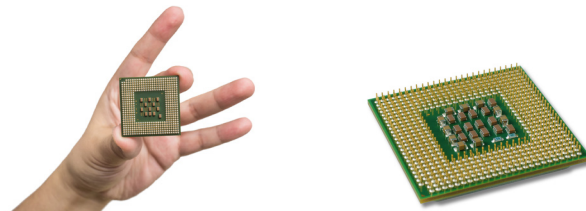
At the interface of an information processing system, only the input and output are visible to the user, whereas the steps that occur inside the system are not. This is because the interface is designed as an abstraction, a simplification of the complex internal workings of the system. Such internal workings are the topic of interest in this lesson.

THE PROCESSOR

In order for a system to generate a response, it must be able to process the information it has gathered. The system must be able to change input to output. This is the task of the processor. The processor is that part of the system dedicated to changing input to output, thereby allowing the system to respond.

THE PROCESSOR IS THE PART OF THE SYSTEM THAT CHANGES INPUT TO OUTPUT.

Just as the user interface is both hardware and software, in the context of processing, we can broadly state that the processor is the hardware through which the signals move and are operated on, while the stored programs are the software that determines the order in which these operations happen and the components of the processor that will perform them.



A typical desktop CPU package measures about 4-5 square centimeters. However, the size of the actual processor chip inside the package is much smaller, typically only a few square millimeters or less.

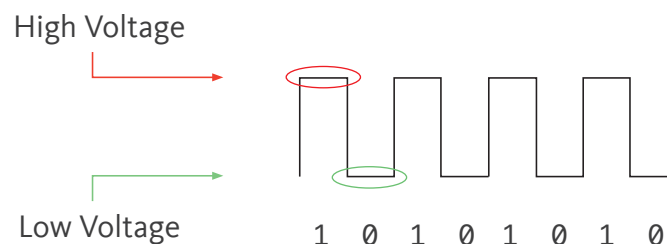
DIGITAL ELECTRONIC SIGNALS

Inside the computer, information is carried as electrical signals. These signals travel through wires arranged in groups of 64 or 32 parallel wires, depending on the computer, with each wire carrying 1 signal or 1 bit of information. A computer with a 64-bit architecture uses 64 parallel wires, while a computer with a 32-bit architecture uses 32 parallel wires, allowing for 64 bits and 32 bits of information to be transmitted simultaneously, respectively.

Each bit is a pulse of electrical potential energy, whose intensity is measured in volts. Therefore, signals can be distinguished based on the intensity of energy they have, into either high voltage or low voltage. To simplify this, a high voltage signal is labeled '1', and a low voltage signal is labeled '0', even though it does not actually have zero volts of energy. Since a signal can only be one of two options, high or low, 1 or 0, we call this type of signal 'binary', which means 'two'. Electrical signals that are binary in nature are referred to as digital electronic signals.

DIGITAL ELECTRONIC SIGNALS ARE BINARY IN NATURE, BEING EITHER HIGH VOLTAGE (1) OR LOW VOLTAGE (0).

A sequence of 8 bits is referred to as a 'byte', whereas the term 'word' is used to refer to the maximum number of bits that can be transmitted simultaneously. In a 32-bit processor, a 'word' is 32 bits long, whereas in a 64-bit processor, a 'word' is 64 bits long.



This is an 8-bit sequence of signals with alternating high and low voltages. The byte can be represented in binary as a series of alternating 1's and 0's: 10101010.

LOGIC CIRCUITS

A logic circuit is a collection of electronic components, mainly transistors, connected by wires in specific configurations. The configuration of transistors determines the operation the logic circuit can perform. Changing the configuration changes the operation. Therefore, a logic circuit that performs addition operations would have a different configuration of transistors from a circuit that performs multiplication operations.

A LOGIC CIRCUIT IS A COLLECTION OF ELECTRONIC COMPONENTS DESIGNED TO PERFORM A PARTICULAR OPERATION.

Processing occurs when digital electronic signals pass through a logic circuit. For example, the adder is a logic circuit that performs an addition operation. It receives two input signals, adds them and then outputs one signal, their sum. There are other logic circuits in the processor, such as the multiplier, comparator and multiplexer.

LOGIC CIRCUIT	PURPOSE
ADDER	Adds 2 inputs
MULTIPLIER	Multiplies 2 inputs
SHIFTER	Shifts the bits of 1 input by 1 or more places
COMPARATOR	Compares 2 inputs as equal, less or greater than
MULTIPLEXER	Selects 1 from several inputs

THE OPERATION A LOGIC CIRCUIT PERFORMS IS DETERMINED BY THE CONFIGURATION OF ITS COMPONENTS.

EXAMPLE 4.1

Let's assume there is a small processor with a 4-bit architecture instead of the more common 64 bits. This hypothetical processor has 4 logic circuits: the adder, multiplier, shifter and comparator.

Let's also assume that the processor receives two 4-bit signals as input, 0110 and 0001. If these two inputs pass through the adder, the output would be their sum, 0111.



Two input signals, 0110 and 0001, pass through an adder logic circuit. The circuit performs the addition operation and generates an output which is the sum, 0111. Note that addition follows the same rules in binary as it does in decimal.

	0	1	1	0
+	0	0	0	1
	0	1	1	1

Now, let's pass the same 4-bit signals, 0110 and 0001, through the multiplier, the output would be their product, 0110. This happens because the logic circuits are arranged in a manner that performs a multiplication operation rather than an addition.



The same two input signals, 0110 and 0001, pass through a multiplier logic circuit. The circuit performs the multiplication operation and generates an output which is the product 0110. Note that multiplication also follows the same rules in binary as it does in decimal. In this case, any number multiplied by 1 is itself.

	0	1	1	0
×	0	0	0	1
	0	1	1	0

On the other hand, the shifter logic circuit would handle the two inputs differently. The first input, 0110, would be considered the signal on which it would perform the shift operation. The second input, 0001, would be the number of places that would be shifted, which in this case is 1 place to the left.



The same two input signals, 0110 and 0001, pass through a shifter logic circuit. The logic circuit performs the shift operation on the first input by shifting its bits by the number of places indicated by the second input, which in this case is 1. The left-most bit is dropped, and a 0 is added as the right-most bit.

In	0	1	1	0
	1	1	0	
Out	1	1	0	0

The comparator accepts two signals but can output 1 of 3 possible outputs: 1) greater than, 2) less than or 3) equal to. It will generate an output signal based on the comparison of the first signal with the second signal. In our small 4-bit processor, we will assign a specific sequence of bits that will be interpreted as greater than, less than or equal to, 1111, 1001 or 0000, respectively.



The same two input signals, 0110 and 0001, pass through a comparator logic circuit. The logic circuit compares them. Since the first signal is greater than the second, $0110 > 0001$, the comparator will output the signal for greater than, 1111.

>	1	1	1	1
=	1	0	0	1
<	0	0	0	0

ALU

The arithmetic logic unit (ALU) is that part of the processor responsible for executing instructions. It contains all the logic circuits needed to perform the operations required for executing instructions.

THE ALU IS THAT PART OF THE PROCESSOR RESPONSIBLE FOR PERFORMING OPERATIONS ON DATA.

An instruction typically specifies the operation that needs to be performed. However, for an operation to function, it requires operands or information to work on. The instruction must provide this information either as data or as an address. Data can be provided immediately in the instruction itself, or the instruction can provide a source address at which the data can be found.

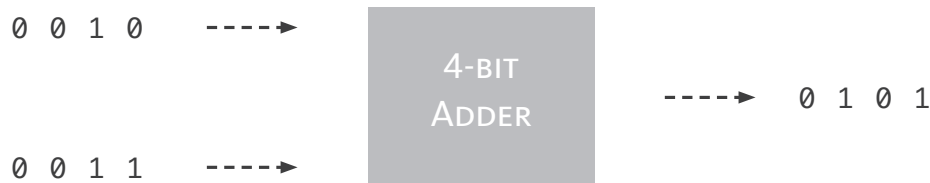
EXAMPLE 4.2

The instruction to add two numbers must specify that the operation to be performed is an addition. It must also specify the two numbers that are to be added, the operands. Let's assume that the numbers to be added are 2 and 3. In this case, the instruction in human-readable language would be:

```
ADD 2,3
```

Notes

Learning the concepts that underpin programming is more critical than learning the programming language itself. Pseudocode is used for that purpose. Written in English, it allows the student to focus on thought processes rather than syntax and semantics.

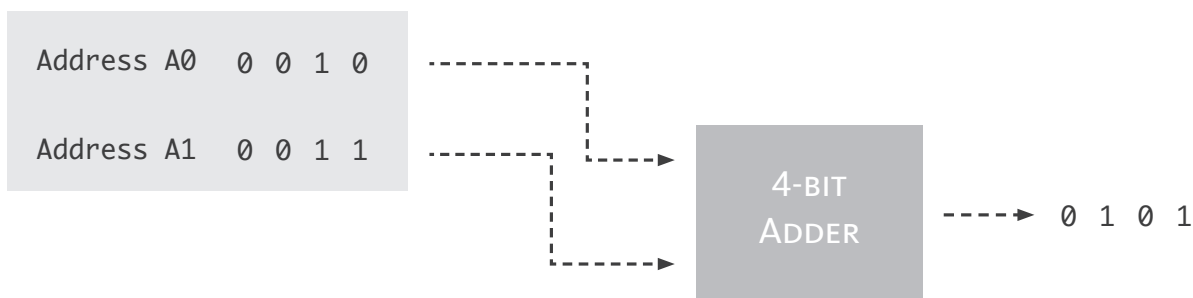


The instruction specifies that the input signals should be 2 and 3, which in binary are represented as 0010 and 0011, respectively. The output from the adder logic circuit would be 5, 0101.

			1	
	0	0	1	0
+	0	0	1	1
	0	1	0	1

Alternatively, the numbers 2 and 3 may already be in the computer and are located at Address A0 and Address A1, respectively. In this case, the instruction would provide the processor with the addresses from where to obtain this data.

ADD A0,A1



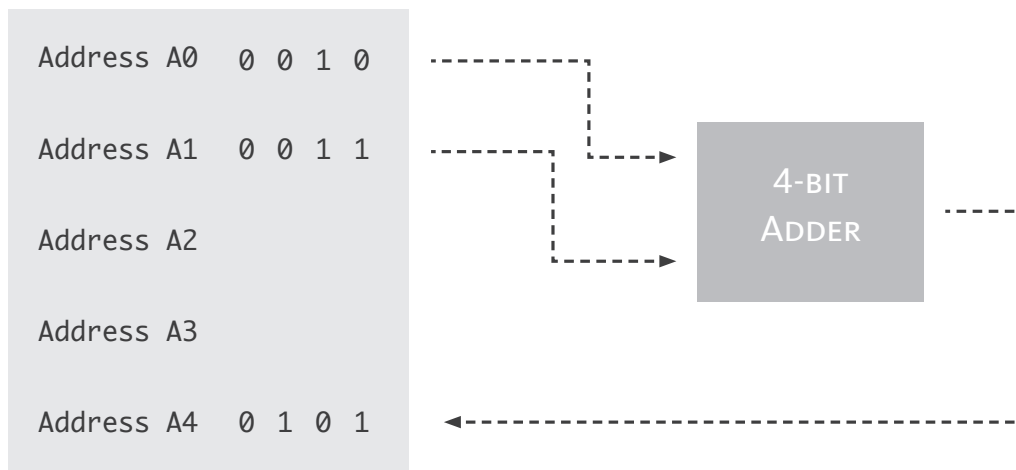
In this case, the instruction provides the data by referring the processor to the source addresses at which the data can be found, Address A0 and Address A1. The data is fetched and fed through the adder logic circuit.

			1	
	0	0	1	0
+	0	0	1	1
	0	1	0	1

However, we still need to figure out where the ALU should send the output of its logic circuits. Just like the ALU has a source for its information, it must

also have a destination. Therefore, the instruction must specify, in addition to a source address, a destination address - the address to which the output should be sent.

ADD A4: A0,A1



The instruction specifies the destination address first, followed by a colon. The two source addresses, Address A0 and Address A1 follow the colon. The processor provides the data from the two source addresses to the adder logic circuit and then delivers the output to the specified address, Address A4.

				1
	0	0	1	0
+	0	0	1	1
	0	1	0	1

Since every instruction must be processed by a logic circuit and the ALU can fit only so many logic circuits, that means that the number of instructions one ALU can execute is limited. In fact, despite the many tasks we perform using our computers, from preparing essays to playing games and watching videos, all these tasks are carried out using a limited set of instructions.

THE INSTRUCTION SET OF A PROCESSOR IS THE SET OF ALL INSTRUCTIONS IT CAN PERFORM.

When designing a processor, engineers first determine the set of instructions the processor needs to execute. Then, they determine the operations needed to execute each instruction and select the logic circuits that perform each operation. Finally, they organize the logic circuits in such a way that allows the operations to be performed in the correct order.

CONTROL

The control unit, CU, is an integral part of the processor. The CU is a collection of logic circuits whose main purpose is to direct the flow of signals through the processor and other parts of the computer. It does so by: 1) decoding instructions to identify the operations that need to be performed, 2) generating control signals to activate the logic circuits that perform these operations and 3) coordinating the operation of the different components of the computer.

In order for the CU to accomplish its purpose, it must be able to identify the operation that needs to be performed; therefore, every instruction in a processor's instruction set must have a unique operation code, opcode. The CU uses this opcode to determine the logic circuits it must activate.

THE CONTROL UNIT DIRECTS THE FLOW OF SIGNALS THROUGH THE PROCESSOR AND COMPUTER.

EXAMPLE 4.3

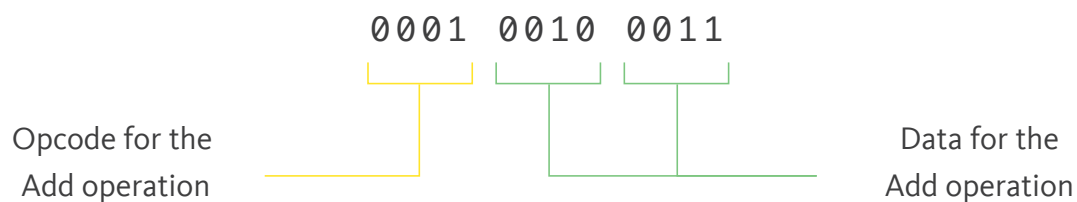
Going back to our 4-bit processor and its four logic circuits, we can create a binary sequence to represent each of the four operation codes required.

Add	0 0 0 1
Multiply	0 0 1 0
Shift	0 0 1 1
Compare	0 1 0 0

Let's revisit the Add instruction that delivers two immediate operands for processing, namely 2 and 3, or 0010 and 0011, respectively. In its human-readable form, it reads:

ADD 2,3

In machine language, using binary signals, this instruction is 12 bits in length and reads:



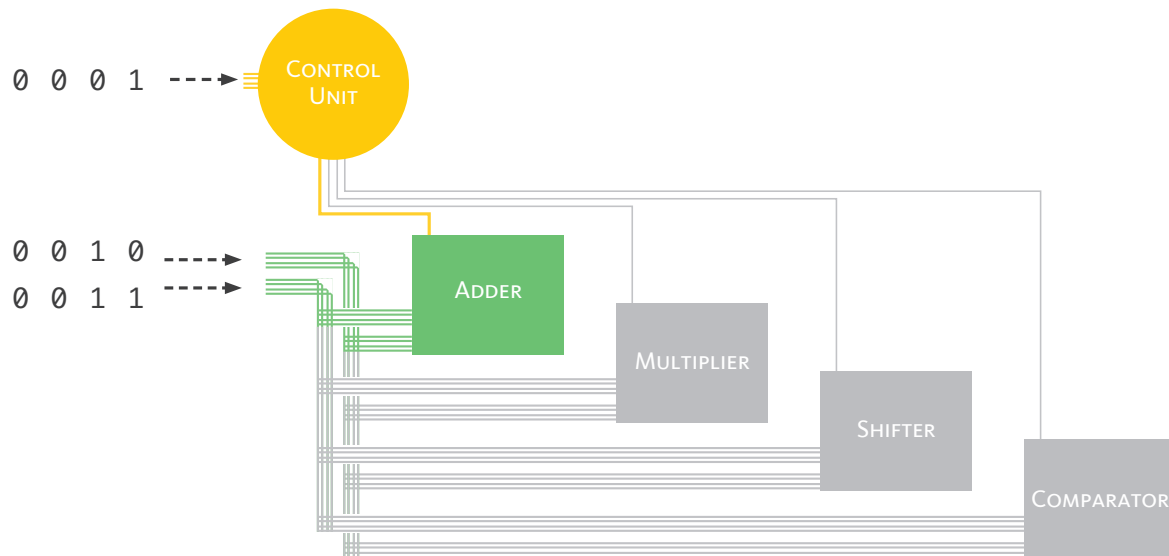
On a side note, this means for our hypothetical small computer to work, 4 bits is not a large enough architecture, and we need to upsize by adding 8 more lines to handle 12 bits instead of 4 bits.

When the instruction is fetched, it is split into its 3 fields: the 4-bit opcode and two 4-bit sequences that represent data. The opcode is sent to the CU for decoding, and, accordingly, a control signal is sent to activate the appropriate logic circuit to which the data is delivered.

MACHINE LANGUAGE IS IN THE FORM OF ELECTRONIC SIGNALS AND IS WRITTEN FOR HUMANS AS ZEROS AND ONES.

Notes

Information exists in two main layers: the logical layer and the physical layer. The logical layer makes sense to us and is represented in the form of human-readable instructions and data. The physical layer is the signals and the logic circuits that process them. The physical layer is the implementation of the logical layer.

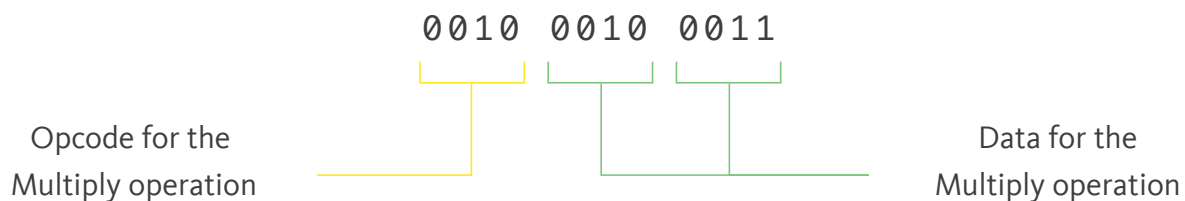


The CU has a control line to each of the four logic circuits it must manage. Once the instruction is fetched, it is split. The opcode `0001` is sent to the CU that interprets it as an Add operation, and then proceeds to send an activation control signal to the adder logic circuit. The activated adder can now receive the two items of data, `0010` and `0011`.

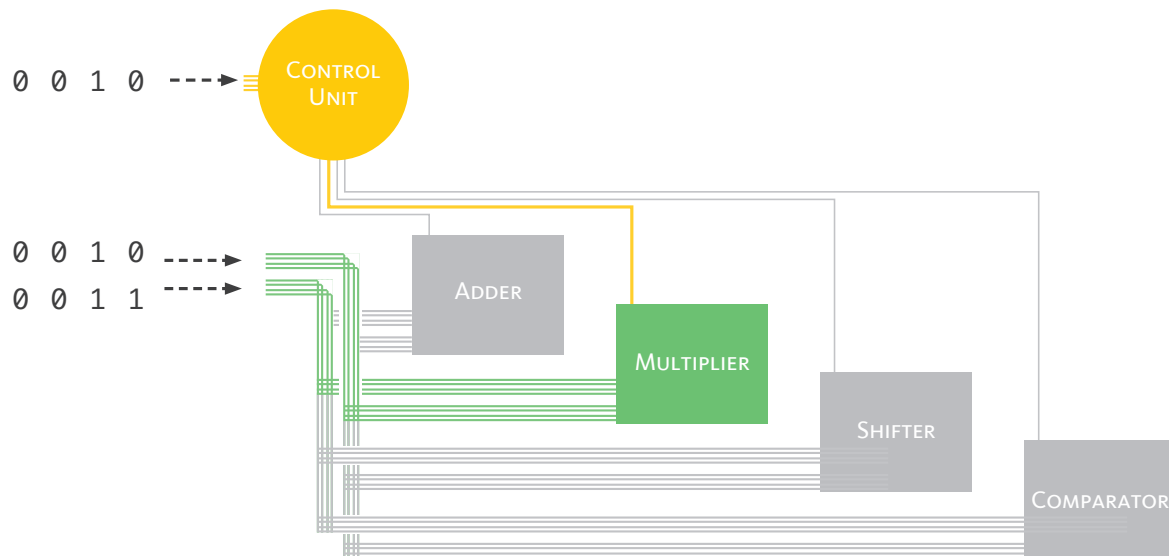
Suppose the Add instruction is changed to a Multiply instruction keeping the same immediate operands, namely 2 and 3, or `0010` and `0011`, respectively. In its human-readable form, the instruction reads:

MULTIPLY 2,3

The opcode for the Multiply operation is `0010`. In machine language, the 12-bit binary instruction reads:



As with every instruction, the instruction is fetched, then it is split into its respective fields. The leftmost field is sent to the CU for decoding, a process that identifies and activates the required components of the computer. The remaining two fields are treated as operands and sent to the activated logic circuit for processing.



In this example, the instruction is for Multiply operation whose opcode is 0010. The 12-bit binary instruction is split. The 4-bit opcode is sent to the CU, which then activates the multiplier logic circuit by sending a control signal. The activated logic circuit can then receive the two items of data, 0010 and 0011.

IN MACHINE LANGUAGE, THE INSTRUCTION IS DIVIDED INTO FIELDS, EACH REPRESENTING A PIECE OF INFORMATION.

The previous hypothetical instructions contained immediate operands, which are data immediately delivered to the ALU for processing. However, not all instructions necessarily contain immediate data. In many cases, instructions involve source addresses that indicate the locations where the data is stored. One of the components that hold such data waiting to be processed is the register file.

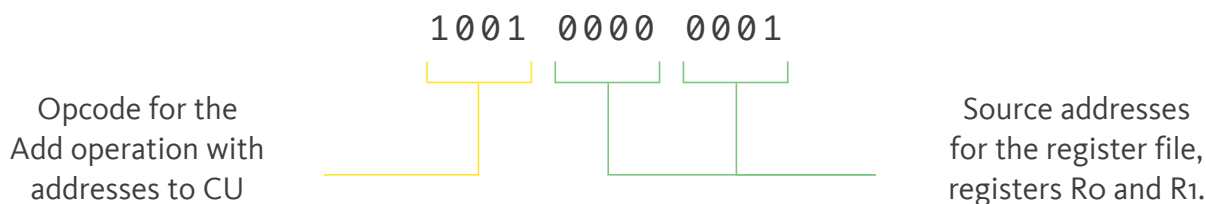
A register is a small and fast memory component within the processor. A processor typically has several registers that are grouped into one working unit referred to as the register file.

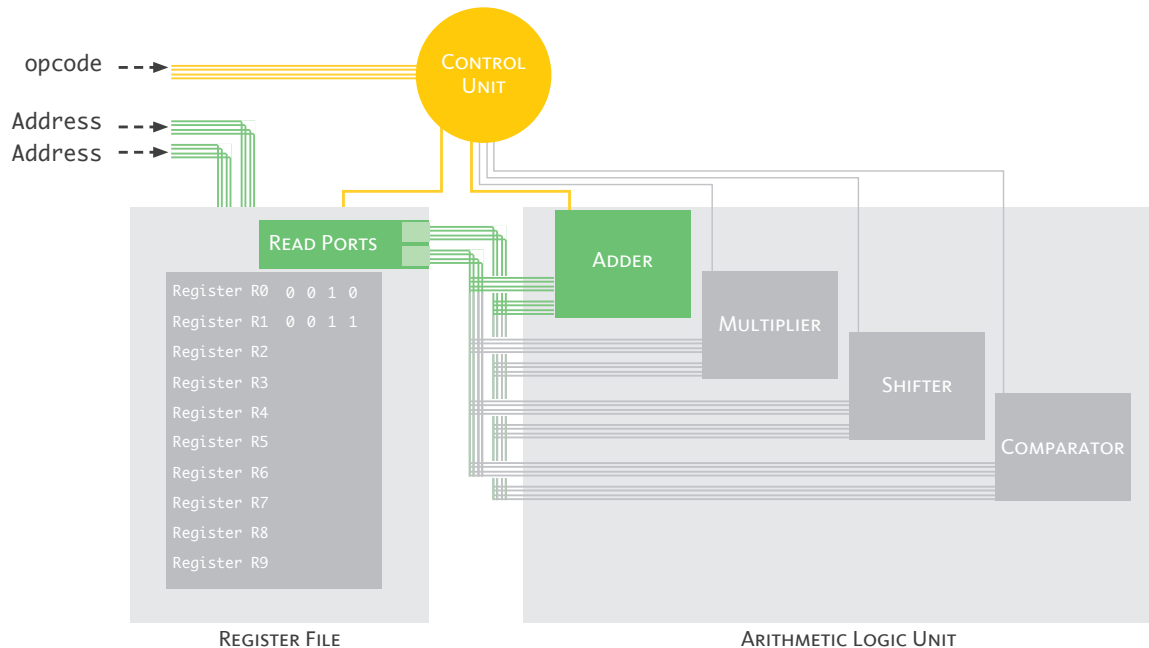
THE REGISTER FILE IS A GROUP OF SMALL AND FAST MEMORY COMPONENTS LOCATED IN CLOSE PROXIMITY TO THE ALU.

Like other components of the processor, the register file is managed by the CU. For instructions that contain source addresses rather than immediate data, the opcode is different. This allows the CU to distinguish between the need to activate the register file and not. When the CU decodes the opcode and identifies that the operation requires the services of the register file, it treats the following two fields as addresses.

A fundamental concept in computing is that any sequence of signals is interpreted based on the context in which it is presented for execution. So if the opcode is for adding immediate data, the following fields are interpreted as numbers. Whereas, if the opcode is for adding the data at source addresses, the following fields are interpreted as addresses.

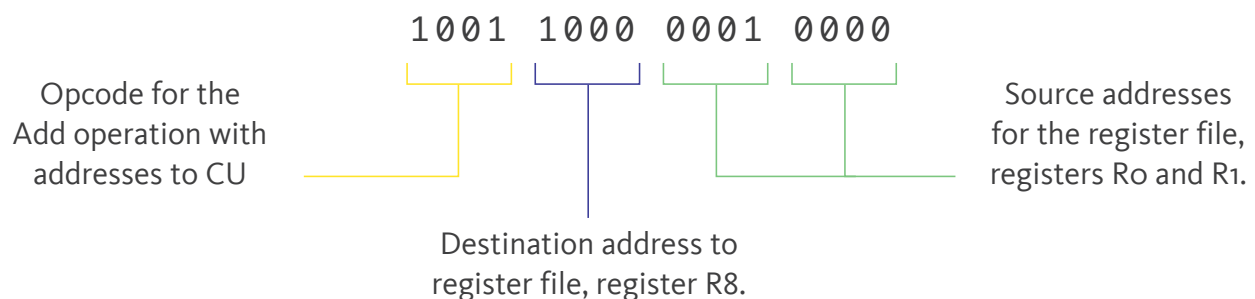
THE INTERPRETATION OF ANY SIGNAL DEPENDS ON THE CONTEXT WITHIN WHICH IT IS BEING EXECUTED.





The instruction is split into 3 fields: the opcode and the two source addresses. The opcode is sent to the control unit, where it is decoded. The CU signals the register file to accept the two addresses as source addresses and to load their contents to the read ports. The adder can then read the data and perform the addition operation.

So far, in our hypothetical processor, we have not considered that data must go somewhere after processing. Once an operation like addition is performed, the result needs to be stored for further use or future instructions. To specify where the output should be stored, the instruction includes a destination address field. This field points to a specific register where the result of the operation will be held.

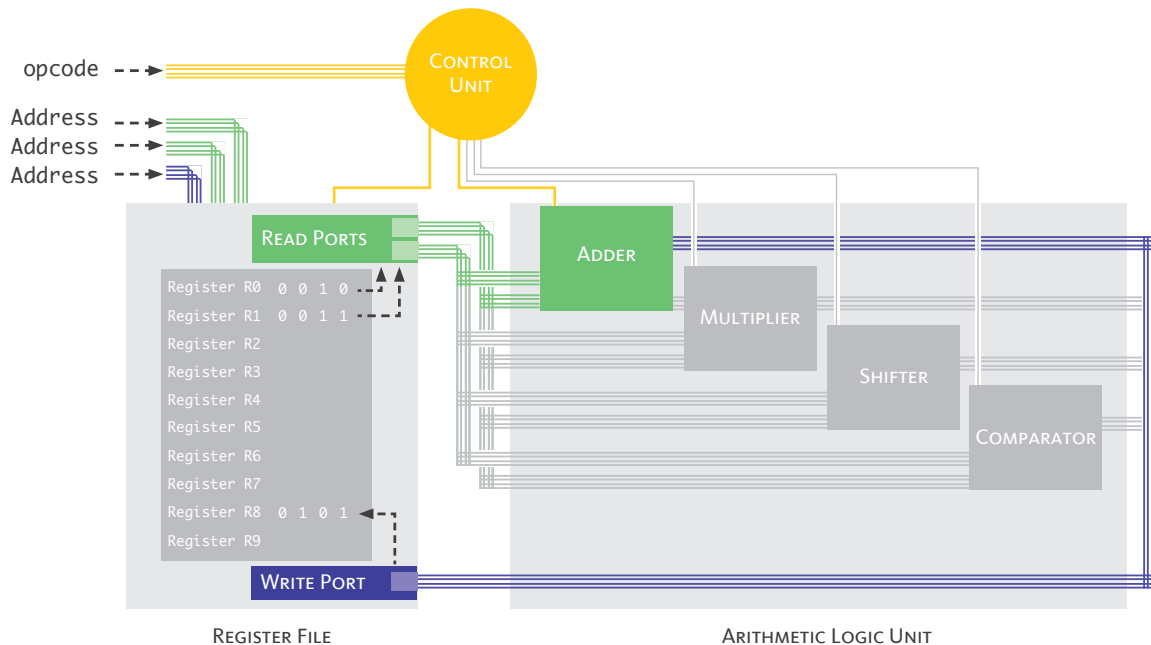


This 16-bit binary sequence, 1001 1000 0001 0000, means: “add the contents of registers R0 and R1 and place the sum in register R8.” In human-readable form, it is transcribed as:

```
ADD R8: R0,R1
```

The destination address, like the source address, typically represents a register number that identifies a specific register within the register file. After decoding the instruction, the control unit signals the register file to write the result into the register specified by the destination address.

By having a designated destination address, the processor ensures that the result is properly stored and can be accessed by subsequent instructions or utilized in further computations.

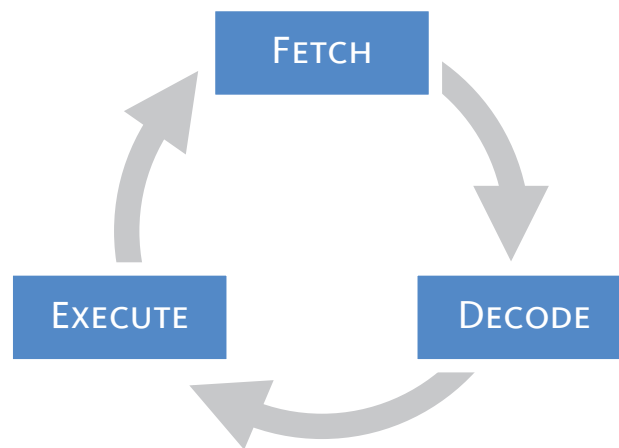


The instruction is split into 4 fields: an opcode, two source addresses and a destination address. The CU signals both the register file and the Adder in the ALU. The register file places the contents of the registers indicated by the source addresses in the read ports. The Adder reads the data, performs the operation and sends the sum back to the register file, which is placed in the register indicated by the destination address.

It is important to note that this lesson reveals the interface between the logical layer, represented in human language, and the physical layer of machine language, represented in digital electronic signals and implemented by logic circuits.

THE INSTRUCTION CYCLE

As long as a processor is working, it is processing instructions one after the other. It fetches an instruction, decodes it, executes it and then fetches the next instruction to repeat the process over and over again. This is referred to as the instruction cycle. Each step is referred to as a stage. Therefore, the instruction cycle is composed of three basic stages: fetch, decode and execute.



THE INSTRUCTION CYCLE IS A SERIES OF REPEATED STAGES: FETCH, DECODE AND EXECUTE.

SUMMARY

While this lesson presented a hypothetical processor, the concepts presented apply to real-life processors.

A computer processes signals referred to as digital electronic signals. These signals are binary in nature in that they can only be one of two values, either high voltage (1) or low voltage (0). Digital electronic signals constitute what is referred to as machine language.

The processor is that part of the computer system that changes input into output. The processor contains logic circuits, which are responsible for processing. Each logic circuit is a collection of electronic components that have a particular arrangement designed to perform a particular operation. The operation performed by a logic circuit is determined by the configuration of its components.

Examples of logic circuits are: adder, multiplier, shifter, comparator and multiplexer.

The processor, also known as the central processing unit (CPU), has three main parts: arithmetic logic unit (ALU), control unit (CU) and register file.

The ALU is responsible for performing operations that include: addition, subtraction, multiplication, division, shifting and comparisons.

The CU coordinates and controls the activities of the processor. It fetches instructions, decodes them and generates the necessary control signals, which ensures that instructions are executed in the correct sequence.

The register file is a group of small and fast memory components called registers. It is located in close proximity to the ALU and is used to hold data and addresses needed in the execution of instructions.

The instruction is split into fields that represent the operation code (opcode), data and/or addresses at which data can be found. The leftmost field is the opcode. The interpretation of the remaining fields depends on the opcode.

The instruction set of a computer is the set of all instructions its processor is designed to perform.

The instruction cycle is a series of stages: fetch, decode and execute. The cycle repeats itself with every instruction.

Activity 4.1

Read the following statements, and write on the line provided for each statement 'T' if it is true and 'F' if it is false.

T/F	STATEMENT
<u>F</u>	The processor is not involved in changing input to output.
<u>T</u>	The main units of a processor are the arithmetic logic circuits, control unit and register file.
<u>T</u>	A digital electronic signal is binary in nature.
<u>T</u>	In a 64-bit processor, a 'word' refers to a sequence of 64 bits.
<u>T</u>	The transistor is an important component of a logic circuit.
<u>F</u>	The configuration of components in a logic circuit does not affect the operation it performs.
<u>T</u>	The ALU is responsible for performing arithmetic operations.
<u>F</u>	The CU is responsible for sending control signals to input and output devices.
<u>T</u>	The opcode determines how the remaining signals in the instruction will be interpreted.
<u>T</u>	Fetch, decode and execute are the three stages of the instruction cycle.

Activity 4.2

Complete the sentences below.

1. Changing input to output is the responsibility of the _____.
 - a) memory
 - b) **processor**
 - c) network adapter

2. Performing arithmetic operations is the responsibility of the _____.
 - a) **ALU**
 - b) CU
 - c) Register file

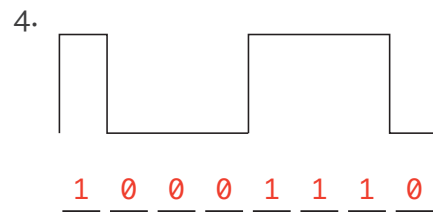
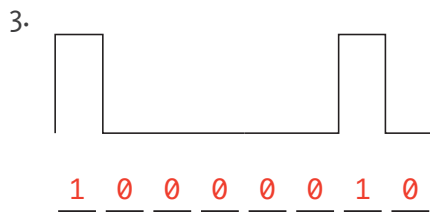
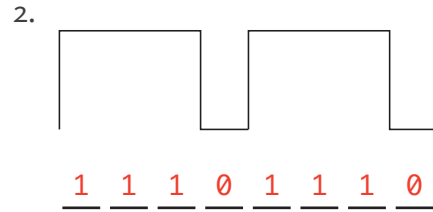
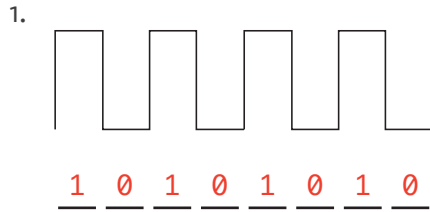
3. Directing the flow of data is the responsibility of the _____.
 - a) ALU
 - b) **CU**
 - c) Register file

4. Which of the following is not true about the register file?
 - a) The register file is a short-term memory unit.
 - b) The register file is a very fast memory unit.
 - c) **The register file is located away from the processor.**

5. Signals going into an information processing system are referred to as:
 - a) decode - fetch - execute
 - b) **fetch - decode - execute**
 - c) execute - decode - fetch

Activity 4.3

Study the digital electronic signals. Encode each in binary form using ones (1) and zeros (0).



Activity 4.4

Perform an addition operation on the 4 pairs of binary numbers.

Note that binary addition is similar to decimal addition, except that $1+1=10$, which requires the 1 to carry forward.

1.

$$\begin{array}{r} 0010 \\ + 0001 \\ \hline 0011 \end{array}$$

2.

$$\begin{array}{r} 0101 \\ + 1000 \\ \hline 1101 \end{array}$$

3.

$$\begin{array}{r} 1100 \\ + 0010 \\ \hline 1110 \end{array}$$

4.

$$\begin{array}{r} 1 \\ 0001 \\ + 0001 \\ \hline 0010 \end{array}$$

5.

$$\begin{array}{r} 11 \\ 0101 \\ + 0101 \\ \hline 1010 \end{array}$$

6.

$$\begin{array}{r} 11 \\ 0011 \\ + 0001 \\ \hline 0100 \end{array}$$

Activity 4.5

Answer the questions below.

1. Which instruction adds two immediates and places the sum in register R8?
 - a) ADD R8: R1,R3
 - b) MULTIPLY R8: 4,5
 - c) **ADD R8: 4,5**

2. Which instruction multiplies data fetched from source addresses and places the product at a destination address?
 - a) **MULTIPLY R5: R4,R0**
 - b) MULTIPLY R8: 4,5
 - c) ADD R8: R1,R3

3. Which instruction is missing a destination address?
 - a) ADD R8: 4,5
 - b) **ADD 2,3**
 - c) ADD R3: R1,R0

4. Which instruction is missing a source address?
 - a) MULTIPLY R6: 0,7
 - b) **MULTIPLY R3: R1**
 - c) MULTIPLY R4: R1,R3

5. Which instruction is missing both source and destination addresses?
 - a) ADD R8: R1,R3
 - b) MULTIPLY R8: 4,5
 - c) **ADD : 4,5**

Activity 4.5

Use the information provided in the tables to translate between human language and the machine of a hypothetical processor to answer the questions.

Opcodes

Add (immediate)	0001
Add (address)	1001
Multiply (immediate)	0010
Multiply (address)	1010

Numbers

0 = 0000	4 = 0100	8 = 1000	12 = 1100
1 = 0001	5 = 0101	9 = 1001	13 = 1101
2 = 0010	6 = 0110	10 = 1010	14 = 1110
3 = 0011	7 = 0111	11 = 1011	15 = 1111

Register Numbers

Register R0 (0000)
Register R1 (0001)
Register R2 (0010)
Register R3 (0011)
Register R4 (0100)
Register R5 (0101)
Register R6 (0110)
Register R7 (0111)

- Which 4-bit binary sequence is designated as the opcode for the operation of adding immediate data?
 - 1001
 - 0001
 - 1101
- Which 4-bit binary sequence represents register R4?
 - 0001
 - 0111
 - 0100
- Which 4-bit binary sequence represents the number 15?
 - 1001
 - 0101
 - 1111

4. Identify the human-readable form of this instruction 1001 0111 0000 0010.
 - a) ADD R4: R1,R2
 - b) MULTIPLY R7: 0,2
 - c) ADD R7: R0,R2

5. Identify the human-readable form of this instruction 0010 0010 0110 0001.
 - a) MULTIPLY R2: 6,1
 - b) MULTIPLY R2: R6,R1
 - c) ADD R2: 6,1

6. Identify the machine language form of this instruction ADD R4: R1,R2.
 - a) 1001 0001 0010 0100
 - b) 1001 0100 0001 0010
 - c) 0001 0100 0001 0010

7. Identify the machine language form of this instruction MULTIPLY R7: 6,7.
 - a) 0010 0110 0111 0111
 - b) 1010 0111 0110 0111
 - c) 0010 0111 0110 0111

8. Write the instruction 1010 0110 0100 0101 in human-readable form.

MULTIPLY R6: R4,R5

9. Write the instruction ADD R3: 4,5 in machine language.

0001 0011 0100 0101

Activity 4.6

Use the information provided in the translation tables of Activity 4.5 to answer the questions.

1. In how many ways can the 4-bit binary sequence 1010 be interpreted? Explain.

It can be interpreted in 3 ways: as an opcode, number and register.

If it is the first field in an instruction, it is interpreted as the opcode for multiply using addresses. If it is the second field, it is interpreted as the register number for the destination register. If it is the third or fourth field, its interpretation will depend on the opcode. An opcode for immediate operations will result in interpreting it as a number, while an opcode for address-related operations will result in interpreting it as a register number.

2. What is the maximum number of integers that can be encoded using 4-bit binary sequences? Can we add more opcodes and registers to our hypothetical processor?

The maximum number of integers that can be encoded is 16, which are the integers 0 to 15. The table in Activity 4.5 lists all the possible 4-bit binary sequences. Alternatively, we can calculate this number. Since there are 4 places and each place can hold one of two values, 0 or 1, we can use $2^4 = 16$.

Yes, we can add opcodes and registers each to a maximum of 16.

Activity 4.7

Answer the questions below.

1. “Instructions provide a context in which signals are interpreted.” Is this statement correct? Explain.

Yes, instructions do provide a context in which signals are interpreted.

Since every instruction has an opcode and the opcode determines how the data is interpreted, then changing the instructions changes the way in which data is interpreted.

2. “I tried to open my Word document using the Excel application, but it didn’t open.” Explain why this statement is true. Does this apply to other applications and files?

In essence, Word and Excel have different ‘instructions’ or ‘opcodes’ for interpreting and handling data, which is why you cannot open a Word document in Excel and expect it to be displayed correctly.

Yes, this principle generally applies to all software applications and their respective files. Each application is specifically designed to work with certain file formats and structures.

